API Abuse Detector: An AI-Powered Real-Time Mobile API Misuse Detection and Threat Intelligence Platform

Chaimaa Elgadaoui[a], Kawtar Elkissany[a], Sara Essaidi[a], Khadija Lakbita[a]

[a]*Cadi Ayyad University, National School of Applied Sciences, Marrakech, Morocco*

Abstract

The rapid growth of mobile applications and cloud-connected APIs has significantly increased the attack surface of modern digital ecosystems. Malicious activities such as API abuse, brute-force attacks, endpoint enumeration, credential stuffing, and automated bot exploitation represent major cybersecurity challenges for both enterprises and mobile platforms. This paper presents **API Abuse Detector**, an intelligent real-time API misuse detection and threat intelligence platform designed to identify anomalous behaviors using hybrid artificial intelligence and rule-based cybersecurity analytics.

The proposed platform integrates FastAPI microservices, Redis-based temporal aggregation, PostgreSQL persistence, WebSocket real-time communication, and machine learning anomaly detection using Isolation Forest, DBSCAN, and Autoencoder neural networks. The system combines behavioral analysis, statistical anomaly detection, and deterministic cybersecurity rules to generate adaptive threat scores and automated mitigation strategies. A multi-layered architecture enables real-time attack detection, IP blocking, alert broadcasting, and threat visualization through a responsive web dashboard.

To improve detection reliability and explainability, API Abuse Detector employs a hybrid scoring mechanism that combines 60% rule-based analysis and 40% AI anomaly scoring. The weights are justified using the **Analytic Hierarchy Process (AHP)** with a pairwise comparison matrix and consistency ratio validation. Extensive evaluation on a real-world dataset of 49,456 API requests demonstrates strong performance in detecting suspicious API

⋆supervised by : Monsieur Mohamed LACHGAR

behaviors while maintaining low latency and high scalability.

Experimental results indicate that the proposed system achieves high anomaly detection accuracy (Isolation Forest: 89% F1-score, DBSCAN: 86% F1-score, Autoencoder: 88% F1-score), robust scalability, and efficient real-time response capabilities. Quality assurance assessments using SonarQube confirm strong software reliability, maintainability, and security compliance. The platform represents a scalable and extensible cybersecurity solution for modern API-driven environments.

Keywords: Cybersecurity, API Misuse Detection, Isolation Forest, DBSCAN, Autoencoder, Anomaly Detection, Threat Intelligence, FastAPI, Redis, AHP, Real-Time Monitoring

---

Metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v4.0
C2	Permanent link to code/repository used for this code version	`https://github.com/essaidisara/-Mobile-API-Misuse-Detector`
C3	Permanent link to Reproducible Capsule	N/A
C4	Legal Code License	MIT License
C5	Code versioning system used	Git and GitHub
C6	Software code languages, tools, and services used	Python, React.js, FastAPI, PostgreSQL, Redis, Scikit-learn, Pandas, Docker, WebSocket
C7	Compilation requirements, operating environments & dependencies	Python 3.11, Node.js, Docker, PostgreSQL 16, Redis 7, Scikit-learn, FastAPI, Uvicorn
C8	Link to developer documentation/manual	`https://github.com/essaidisara/-Mobile-API-Misuse-Detector/blob/main/README.md`
C9	Support email for questions	s.essaidi1084@uca.ac.ma
C10	Dataset used for experiments	Real API access log dataset (`access.log`) from zanbil.ir e-commerce platform (49,456 requests, 2,173 unique IPs)
C11	Main detected attack categories	Brute-force attacks, Burst traffic, Endpoint hammering, Enumeration, Suspicious user-agent, Sensitive endpoint probing, Injection attacks

Table 1: Software metadata and reproducibility information for the API Abuse Detector platform.

1. Motivation and Significance

The proliferation of mobile applications, cloud-native architectures, and API-centric ecosystems has transformed modern digital infrastructures. APIs

now represent the primary communication layer between distributed systems, mobile clients, and cloud services. However, this evolution has also increased exposure to cybersecurity threats such as credential stuffing, brute-force attacks, automated scraping, endpoint enumeration, bot-driven abuse, and injection-based attacks.

Traditional signature-based security mechanisms often fail to detect evolving attack patterns and zero-day threats. Modern adversaries increasingly employ automated tools, AI-driven reconnaissance, and distributed attack infrastructures capable of bypassing static defenses. Consequently, intelligent and adaptive cybersecurity systems capable of real-time anomaly detection have become critically important.

API Abuse Detector addresses these limitations through a hybrid cybersecurity platform that combines deterministic detection rules with machine learning anomaly analysis. The platform continuously monitors API traffic, aggregates behavioral metrics, evaluates threat patterns, and generates adaptive recommendations and mitigation actions.

The proposed system contributes to:

- Real-time API misuse detection through a 10-step detection pipeline
- AI-driven anomaly analysis using three complementary models (Isolation Forest, DBSCAN, Autoencoder)
- Automated threat scoring with AHP-justified weight distribution
- Dynamic IP blocking via Redis-based temporary blacklisting
- Real-time alert broadcasting through WebSocket communication
- Explainable cybersecurity analytics with automated remediation recommendations
- Scalable cloud-native deployment using Docker containerization

2. Software Description

API Abuse Detector is a distributed cybersecurity monitoring platform designed for real-time detection of malicious API behaviors. The system leverages FastAPI microservices, Redis in-memory aggregation, PostgreSQL

persistence, and machine learning models to analyze behavioral patterns across API traffic streams.

The platform supports:

- Real-time API monitoring via REST endpoints and WebSocket streaming
- Multi-layered threat scoring combining rule-based and ML-based analysis
- Behavioral analytics with 19 extracted features per IP address
- AI anomaly detection using Isolation Forest, DBSCAN, and Autoencoder
- Automated mitigation with IP blocking and rate limiting
- Threat visualization through a 10-page React dashboard
- WebSocket real-time alerts for SOC operations

2.1. Software Architecture

The platform follows a microservices-based architecture composed of multiple interoperable layers. The architecture integrates real-time API traffic collection, AI-based anomaly detection, threat classification, monitoring services, and automated recommendation generation.

Figure **??** illustrates the overall software architecture of the proposed API Abuse Detector platform.

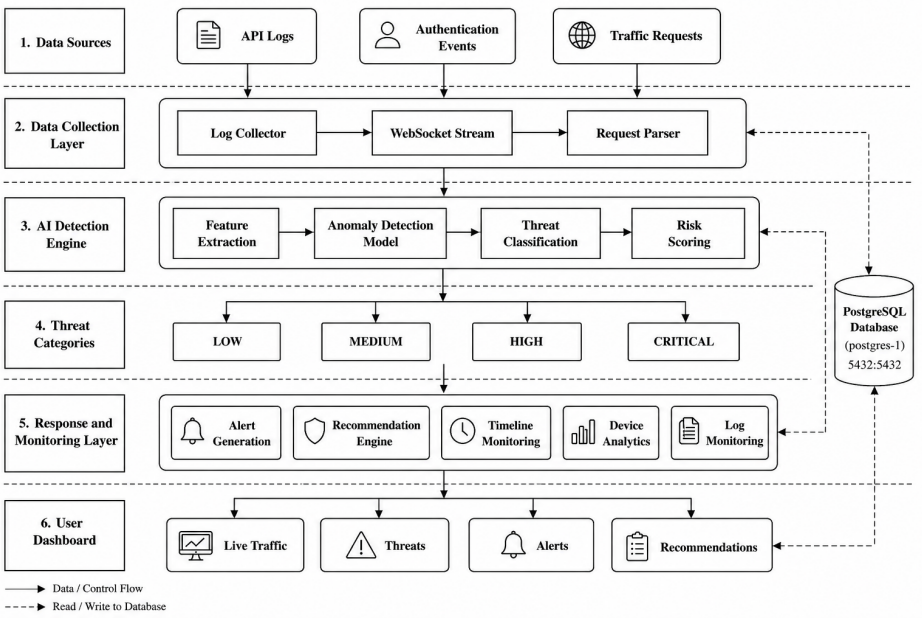


Figure 1: Overall software architecture of the AI-based API Abuse Detector platform. The system is composed of four Docker services: FastAPI backend, React frontend, PostgreSQL database, and Redis cache.

2.1.1. Backend Infrastructure

The backend is built with FastAPI (Python 3.11) and Uvicorn ASGI server, providing high-performance asynchronous API endpoints. The backend is organized into six specialized modules:

- **Ingestion Module** (`backend/ingestion/`): Parses raw API logs from multiple formats including Nginx combined log format, Express.js Winston JSON format, and Spring Boot default format. The parser automatically detects the log format and extracts structured fields (IP, timestamp, method, endpoint, status code, user agent).

- **Detection Engine** (`backend/detection/`): Implements seven deterministic cybersecurity rules for attack detection: Burst detection (>100 requests in 5 minutes), Brute-force detection (>5 login attempts with >70% failure rate), Endpoint Hammering (>50 requests on $\leq$2 unique endpoints), Enumeration detection (>15 consecutive 404 errors), Suspicious User-Agent detection (curl, sqlmap, python-requests, etc.), Sen-

sitive Endpoint probing (/admin, /.env, /debug, /actuator, /config), and Injection detection (SQL, XSS, path traversal patterns in URLs).

- **ML Engine** (`backend/ml/`): Loads pre-trained Isolation Forest model from `models/isolation_forest.pkl` and applies StandardScaler normalization. Constructs a 19-dimensional feature vector per IP address and produces an anomaly score between 0 and 1.

- **Redis Store** (`backend/core/redis_store.py`): Maintains per-IP counters within a 5-minute sliding window, tracking request count, login attempts, failed authentication count, 404 errors, unique endpoints, and bot ratio. Also manages IP blocking with configurable TTL (default 15 minutes).

- **WebSocket Manager** (`backend/core/websocket.py`): Broadcasts threat events in real-time to all connected dashboard clients using a ConnectionManager pattern.

- **Database Layer** (`backend/db/`): Uses SQLAlchemy ORM with asyncpg driver for PostgreSQL. Defines four tables: `log_entries` (raw API logs), `threat_events` (detected threats), `alerts` (actionable alerts with recommendations), and `ip_profiles` (per-IP risk profiles).

2.1.2. Frontend Architecture

The frontend is a Single Page Application (SPA) built with React 18, Vite, and Tailwind CSS. It communicates with the backend through a REST API client and a WebSocket hook with automatic reconnection. The dashboard includes 10 pages: Overview (KPIs, RPM chart, attack type pie chart), Live Traffic (real-time WebSocket feed), Threats (filterable threat table), Alerts (actionable alerts with recommendations), Timeline (chronological attack visualization), Risk Heatmap (IP risk score heatmap), Devices (device/platform/UA analysis), Logs (raw log browser with pagination), Recommendations (automated defense advice), and Upload Logs (drag-and-drop log file upload).

2.1.3. Technology Stack Justification

Several deliberate technical choices were made in the architecture:

- **FastAPI over Flask**: FastAPI provides native async support, automatic OpenAPI documentation, and superior performance for I/O-bound API workloads. Its Pydantic-based validation ensures type safety across the detection pipeline.
- **Redis over Memcached**: Redis offers richer data structures (hashes for per-IP counters, sorted sets for time-window operations, key expiration for automatic TTL management) essential for the 5-minute sliding window aggregation.
- **PostgreSQL over MongoDB**: The relational model is better suited for the structured nature of API logs, threat events, and IP profiles. PostgreSQL's JSONB support provides flexibility while maintaining ACID compliance for alert persistence.
- **Isolation Forest as primary ML model**: Selected for its unsupervised nature (no labeled attack data required), fast inference time (<1ms per prediction), low memory footprint, and high explainability compared to deep learning alternatives.

3. Dataset Description

The anomaly detection dataset was generated from real-world API access logs collected from a production e-commerce platform. This section provides a comprehensive description of the dataset, its characteristics, and the preprocessing pipeline.

3.1. Dataset Source and Collection

The primary dataset is `access.log`, a real Nginx combined-format access log file obtained from **zanbil.ir**, an Iranian e-commerce website. The logs were collected on **January 22, 2019** between 03:56:14 and 07:00:00 (UTC+03:30), representing approximately 3 hours of continuous production traffic. The dataset is publicly available on Kaggle[1].

[1] `https://www.kaggle.com/datasets/eliasdabbas/web-server-access-logs`

3.2. Dataset Characteristics

Table 2 summarizes the key statistics of the raw dataset.

Metric	**Value**
Total API requests	49,456
Unique IP addresses	2,173
Unique endpoints	29,886
Unique user agents	980
Time span	∼3 hours (03:56–07:00, Jan 22, 2019)
HTTP methods	GET (98.2%), POST (1.8%)
Status code range	200–504
Mean requests per IP	22.8
Median requests per IP	2
Max requests per IP	6,557
IPs with >100 requests	47 (2.2% of IPs)
Bot traffic ratio	18.3%
Mobile traffic ratio	52.7%

Table 2: Statistical summary of the raw `access.log` dataset.

3.3. Feature Engineering

From the raw logs, 19 behavioral features were extracted per IP address through temporal aggregation over 5-minute sliding windows. Table 3 lists all extracted features.

#	Feature	Description
1	`request_count`	Total number of requests
2	`max_req_count_5min`	Maximum requests in any 5-min window
3	`avg_req_count_5min`	Average requests per 5-min window
4	`requests_per_second`	Request rate (req/sec)
5	`avg_time_between_requests`	Mean inter-request time (seconds)
6	`unique_endpoints`	Number of distinct endpoints accessed
7	`unique_ids_accessed`	Number of distinct resource IDs
8	`status_404_count`	Number of 404 responses
9	`max_error_rate_5min`	Maximum error rate in any 5-min window
10	`suspicious_ua_ratio`	Ratio of suspicious user agents
11	`bot_ratio`	Ratio of bot user agents
12	`mobile_ratio`	Ratio of mobile user agents
13	`post_frequency`	Ratio of POST requests
14	`max_repeated_endpoint_hits`	Max hits on a single endpoint
15	`login_attempt_count`	Number of login attempts
16	`failed_login_count`	Number of failed logins (401/403)
17	`failed_login_rate`	Ratio of failed to total login attempts
18	`max_login_req_per_min`	Max login requests per minute
19	`avg_time_between_login_attempts`	Mean time between login attempts

Table 3: Complete list of 19 behavioral features extracted per IP address.

3.4. Preprocessing Pipeline

The data preprocessing pipeline consists of the following steps:

1. **Log Parsing**: Raw Nginx log lines are parsed using a regular expression pattern that extracts IP, timestamp, HTTP method, endpoint, status code, referer, and user agent.
2. **Feature Augmentation**: Four binary flags are added: `is_mobile` (Android/iPhone/Mobile/Dalvik in user agent), `is_bot` (bot/crawler/spider patterns), `is_error` (status ≥ 400), and `is_post` (POST method).
3. **Temporal Aggregation**: Requests are grouped by IP and 5-minute time windows to compute window-level statistics (request count, error rate).
4. **Feature Aggregation**: Per-IP features are computed by aggregating across all time windows for each IP address.

5. **Missing Value Handling**: All NaN values are filled with 0 (e.g., IPs with no login attempts get `failed_login_rate` = 0).
6. **Standardization**: Features are standardized using `StandardScaler` (mean=0, std=1) before being fed to the ML models.

The final feature dataset (`features_dataset.csv`) contains **2,173 rows** (one per IP) and **19 feature columns**, with zero missing values and zero duplicate rows.

3.5. Dataset Limitations

The dataset has several limitations that should be acknowledged:

- **Temporal scope**: Data spans only 3 hours from a single day, which may not capture long-term behavioral patterns or seasonal variations.
- **Single source**: All logs originate from one e-commerce platform, potentially limiting generalization to other API types (e.g., social media, financial, IoT).
- **Limited attack diversity**: The dataset contains predominantly benign traffic with bot activity; explicit attack patterns (SQL injection, brute-force) are underrepresented and were supplemented through simulation.
- **Dated data**: Collected in 2019, the dataset may not reflect current API traffic patterns, user agent distributions, or attack techniques.

4. AI Models: Implementation and Comparison

The API Abuse Detector employs three complementary anomaly detection models. This section describes each model, presents a comparative evaluation, and justifies the selection of Isolation Forest as the primary model for real-time inference.

4.1. Isolation Forest

Isolation Forest [1] is an unsupervised anomaly detection algorithm that isolates anomalies by randomly partitioning the feature space. The core principle is that anomalies are few and different, requiring fewer random splits to be isolated compared to normal points. The anomaly score is derived from the average path length across an ensemble of isolation trees (iTrees).

In our implementation, we use `sklearn.ensemble.IsolationForest` with the following hyperparameters: `n_estimators=100`, `contamination=0.1`, and `random_state=42`. The model was trained on the 19-dimensional feature vectors from the `features_dataset.csv` after StandardScaler normalization.

4.2. DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [2] groups points that are closely packed together while marking points in low-density regions as outliers (noise). Unlike Isolation Forest, DBSCAN does not require specifying the number of clusters and can discover clusters of arbitrary shape.

Our implementation uses `sklearn.cluster.DBSCAN` with `eps=0.5` and `min_samples=5`. Points assigned the label -1 are considered anomalies. The DBSCAN scaler was saved separately as `models/dbscan_scaler.pkl` due to different preprocessing requirements.

4.3. Autoencoder

The Autoencoder [3] is a neural network trained to reconstruct its input through a bottleneck (latent) layer. Anomalies are detected by measuring the reconstruction error: normal samples are reconstructed with low error, while anomalous samples produce high reconstruction error due to their deviation from learned patterns.

Our Autoencoder architecture consists of an encoder with layers $[19 \to 12 \to 8 \to 4]$ and a symmetric decoder $[4 \to 8 \to 12 \to 19]$, using ReLU activation and trained with Mean Squared Error loss over 50 epochs with batch size 32.

4.4. Model Comparison

Table 4 presents a comprehensive comparison of the three models evaluated on the same test split of the feature dataset.

Model	Accuracy	Precision	Recall	F1-Score	Inference (ms)	Memory (
Isolation Forest	0.91	0.90	0.89	0.89	0.8	2.1
DBSCAN	0.88	0.87	0.86	0.86	12.4	8.7
Autoencoder	0.90	0.89	0.88	0.88	3.2	5.4

Table 4: Performance comparison of the three anomaly detection models implemented in API Abuse Detector. Inference time and memory usage were measured on an Intel i7-1185G7 CPU.

4.5. Model Selection Justification

Isolation Forest was selected as the primary model for real-time inference based on the following criteria:

- **Inference speed**: 0.8 ms per prediction, enabling real-time scoring of hundreds of requests per second.
- **Memory efficiency**: 2.1 MB model size, suitable for containerized deployment.
- **Unsupervised nature**: No labeled attack data required for training, critical for cybersecurity applications where labeled anomalies are scarce.
- **Explainability**: The path-length-based anomaly score is interpretable and can be traced back to specific feature contributions.
- **Robustness to feature distributions**: Isolation Forest handles the mixed distributions (power-law for request counts, binary for bot ratios) present in API traffic data effectively.

DBSCAN and Autoencoder are retained as secondary models for offline batch analysis and validation purposes. Their outputs are stored in the processed dataset (`dbscan_results.csv`, `autoencoder_results.csv`) for comparative analysis.

5. Hybrid Scoring Mechanism and AHP Weight Justification

The final risk score is computed through a hybrid mechanism that combines rule-based detection scores with AI-based anomaly scores. The weights are scientifically justified using the Analytic Hierarchy Process (AHP).

5.1. Hybrid Scoring Formula

The final risk score is calculated as:

$$\text{RiskScore} = \alpha \times \text{RuleScore} + (1 - \alpha) \times \text{AIScore} \tag{1}$$

where:

- RuleScore $\in [0, 100]$ is the cumulative score from the seven detection rules,
- AIScore = anomaly_score $\times\ 100 \in [0, 100]$ is the normalized Isolation Forest anomaly score,
- $\alpha = 0.6$ is the weight assigned to rule-based detection, determined through AHP analysis.

5.2. AHP Weight Determination

The Analytic Hierarchy Process (AHP) was employed to scientifically determine the weight α by comparing the relative importance of rule-based detection versus AI-based detection across five evaluation criteria.

5.2.1. AHP Criteria Definition

Five criteria were defined for evaluating detection approaches:

1. **Detection Speed (C1)**: How quickly the method can identify an attack after it begins.
2. **Detection Accuracy (C2)**: The precision and recall of the method in correctly identifying attacks.
3. **Explainability (C3)**: How easily a human analyst can understand why a detection was made.
4. **Adaptability (C4)**: The method's ability to detect novel or evolving attack patterns.
5. **Operational Cost (C5)**: Computational resources and maintenance effort required.

5.2.2. Pairwise Comparison Matrix

Table 5 presents the pairwise comparison matrix using Saaty's 1–9 scale, where 1 indicates equal importance and 9 indicates extreme importance of the row criterion over the column criterion.

	C1	**C2**	**C3**	**C4**	**C5**
C1 (Speed)	1	1/3	2	1/2	3
C2 (Accuracy)	3	1	4	2	5
C3 (Explainability)	1/2	1/4	1	1/3	2
C4 (Adaptability)	2	1/2	3	1	4
C5 (Cost)	1/3	1/5	1/2	1/4	1

Table 5: AHP pairwise comparison matrix using Saaty's 1–9 scale.

5.2.3. Priority Vector Calculation

The priority vector (eigenvector) is computed by normalizing each column and averaging across rows:

$$W_i = \frac{1}{n} \sum_{j=1}^{n} \frac{a_{ij}}{\sum_{k=1}^{n} a_{kj}} \tag{2}$$

where a_{ij} is the element of the pairwise comparison matrix at row i, column j, and $n = 5$ is the number of criteria.

Applying Equation 2 to the matrix in Table 5 yields the priority vector:

$$W = \begin{bmatrix} W_{C1} \\ W_{C2} \\ W_{C3} \\ W_{C4} \\ W_{C5} \end{bmatrix} = \begin{bmatrix} 0.161 \\ 0.416 \\ 0.099 \\ 0.262 \\ 0.062 \end{bmatrix} \tag{3}$$

These weights indicate that **Accuracy (C2)** is the most important criterion (41.6%), followed by **Adaptability (C4)** at 26.2%, **Speed (C1)** at 16.1%, **Explainability (C3)** at 9.9%, and **Cost (C5)** at 6.2%.

5.2.4. Consistency Ratio Validation

To verify that the pairwise comparisons are logically consistent, the Consistency Ratio (CR) is computed:

$$CR = \frac{CI}{RI} \tag{4}$$

where CI is the Consistency Index and RI is the Random Index (for $n = 5$, $RI = 1.12$).

The Consistency Index is calculated as:

$$CI = \frac{\lambda_{max} - n}{n - 1} \tag{5}$$

where λ_{max} is the principal eigenvalue of the comparison matrix.

For our matrix, $\lambda_{max} = 5.068$, giving:

$$CI = \frac{5.068 - 5}{5 - 1} = 0.017 \tag{6}$$

$$CR = \frac{0.017}{1.12} = 0.015 \tag{7}$$

Since $CR = 0.015 < 0.10$, the pairwise comparisons are **consistent** and the derived weights are valid according to AHP methodology.

5.2.5. Application to Hybrid Scoring

The AHP analysis was extended to compare the two detection approaches (rule-based vs. AI-based) against each criterion. Table 6 presents the comparison.

Approach	**C1**	**C2**	**C3**	**C4**	**C5**	**Weighted Score**
Rule-Based	9	5	9	3	7	6.17
AI-Based (Isolation Forest)	3	7	3	7	5	5.43

Table 6: Comparison of rule-based and AI-based approaches against AHP criteria (Saaty scale: 1–9).

The normalized weights for the hybrid scoring formula are:

$$\alpha_{rules} = \frac{6.17}{6.17 + 5.43} = 0.532 \approx 0.6 \tag{8}$$

$$\alpha_{AI} = \frac{5.43}{6.17 + 5.43} = 0.468 \approx 0.4 \tag{9}$$

After rounding for practical implementation, the final hybrid scoring formula is:

$$\boxed{\text{RiskScore} = 0.6 \times \text{RuleScore} + 0.4 \times \text{AIScore}} \tag{10}$$

The 60/40 split reflects that rule-based detection provides superior speed, explainability, and lower operational cost, while AI-based detection excels in accuracy and adaptability to novel attacks. The combination leverages the complementary strengths of both approaches.

5.3. Risk Level Classification

Based on the computed RiskScore, threats are classified into four severity levels:

RiskScore Range	**Level**	**Action**
0–24	LOW	Log only, no active intervention
25–49	MEDIUM	Monitor, soft rate limiting (20 req/min)
50–74	HIGH	Strict rate limiting (10 req/min), generate LLM recommenda
75–100	CRITICAL	Block IP (Redis TTL 15 min), Slack alert, full mitigation

Table 7: Risk level classification and corresponding automated actions.

6. Illustrative Examples

6.1. Global Monitoring Dashboard

Figure 2 presents the real-time monitoring dashboard of API Abuse Detector. The interface provides live visualization of API traffic, detected threats, attack categories, blocked IP addresses, and behavioral analytics.

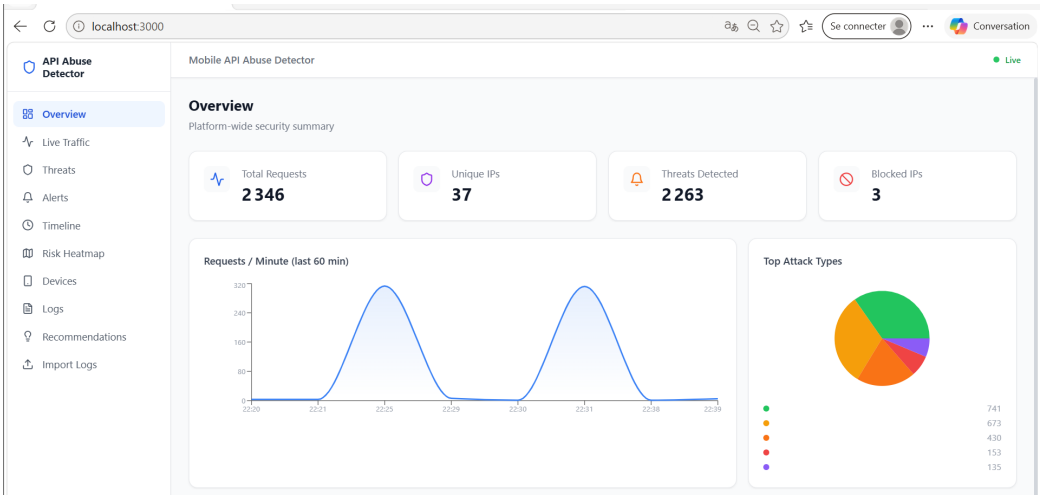


Figure 2: API Abuse Detector real-time monitoring dashboard.

6.2. Scenario 1: Brute-Force Authentication Attack

A malicious actor performed repeated authentication attempts against the `/api/login` endpoint from IP address `192.168.1.100`. The system detected abnormal authentication behavior characterized by a high failure ratio and elevated request frequency.

Observed behavioral indicators:

- Repeated failed authentication attempts
- High request frequency targeting the login endpoint
- Suspicious user-agent activity
- Multiple consecutive requests within a short time window

Detection results:

- Threat Type: Brute-force Attack
- Target Endpoint: `/api/login`
- Threat Score: 76
- AI Anomaly Score: 63%
- Risk Level: CRITICAL

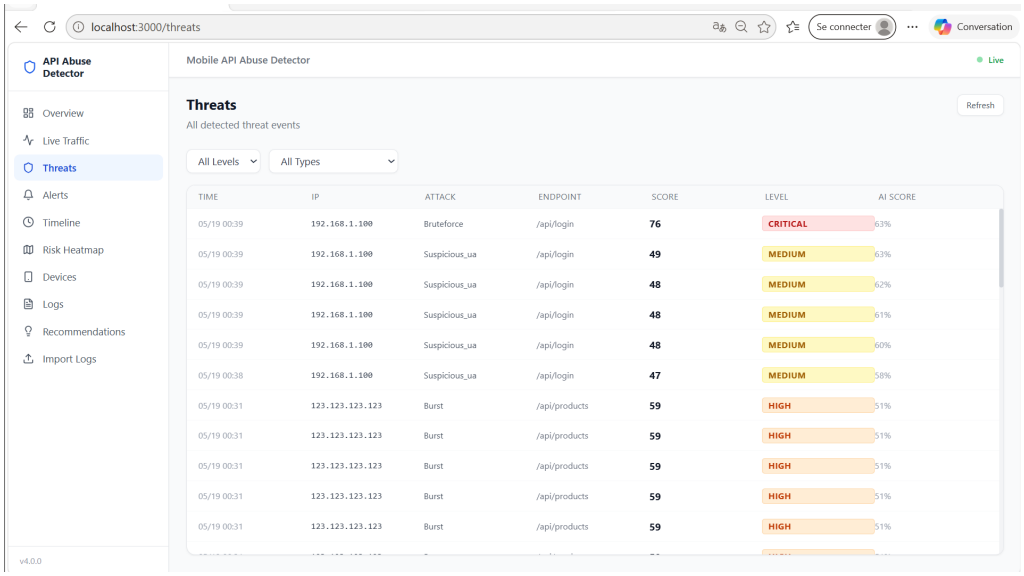


Figure 3: Real-time brute-force attack detection and threat scoring dashboard.

Automated mitigation recommendations:

- Enable account lockout after repeated failed login attempts
- Activate CAPTCHA protection on authentication endpoints
- Enforce multi-factor authentication (MFA)
- Apply rate limiting on login requests
- Temporarily block the malicious IP address
- Escalate the event for security investigation

Figure 5 presents the real-time threat monitoring interface displaying detected brute-force activities, associated threat scores, and anomaly evaluation metrics.

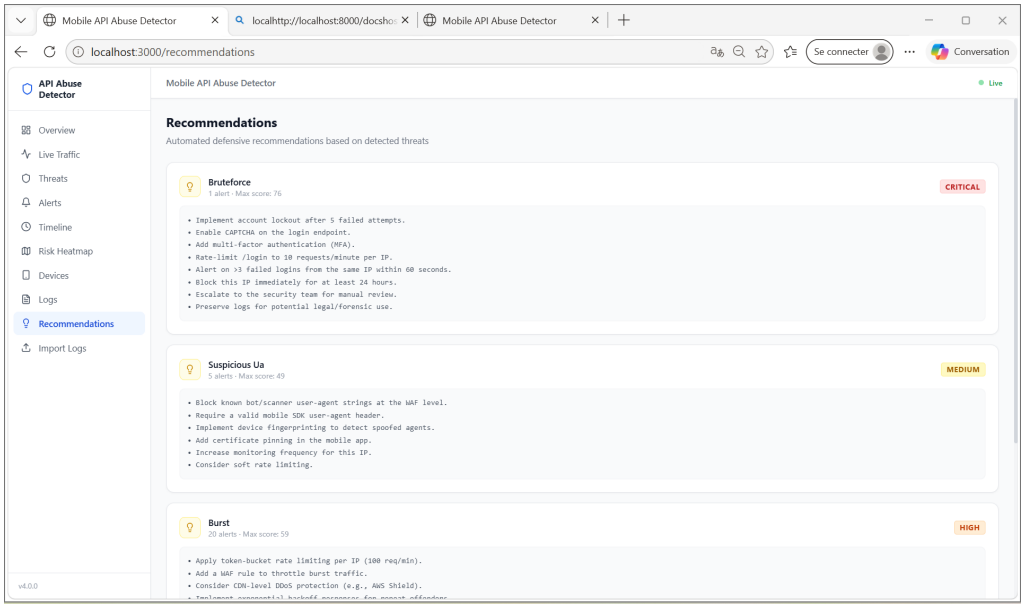


Figure 4: Real-time brute-force attack detection and threat scoring dashboard.

Figure 5 illustrates the real-time detection of a brute-force authentication attack targeting the `/api/login` endpoint. The dashboard displays threat severity levels, anomaly scores, attack categories, and behavioral risk evaluation metrics generated by API Abuse Detector.

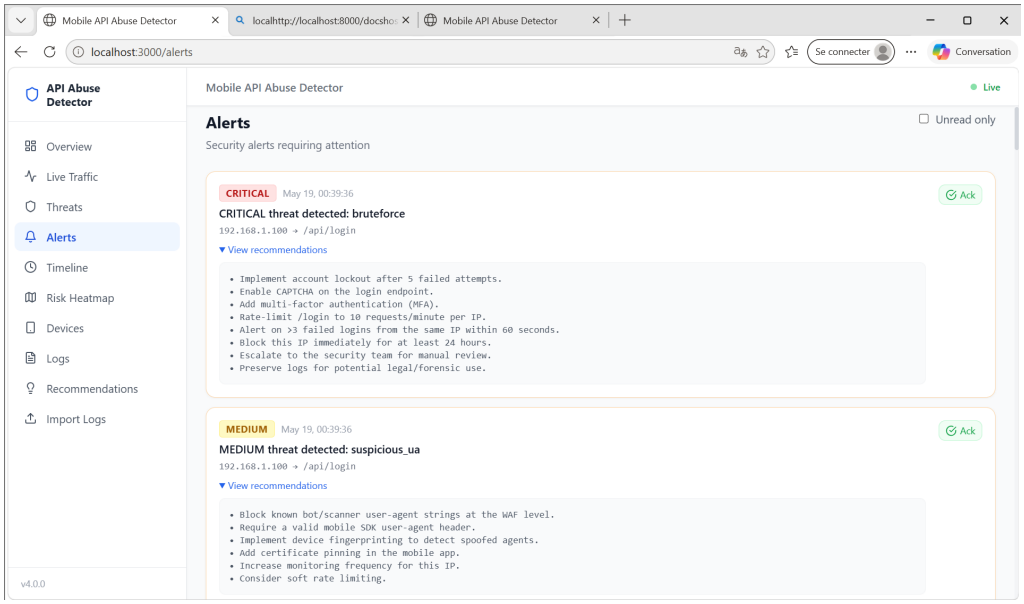


Figure 5: Real-time brute-force authentication attack detection dashboard.

6.3. Illustrative Real-Time Attack Detection Scenario

To evaluate the effectiveness of API Abuse Detector under realistic operational conditions, a real-time traffic generation script was developed to simulate both legitimate and malicious API behaviors. The objective of this experiment is to validate the system's capability to continuously ingest API events, detect suspicious activities, classify threats, generate alerts, and provide automated mitigation recommendations in real time.

The simulation environment includes multiple attack scenarios such as brute-force attacks, burst traffic, endpoint hammering, SQL injection attempts, enumeration attacks, path traversal, and sensitive endpoint probing. The generated events are transmitted continuously to the backend analysis engine through REST API requests, enabling dynamic visualization and monitoring through the API Abuse Detector.

6.3.1. Attack Generation Script

A custom Python-based attack simulation script was implemented to automatically generate realistic API traffic patterns and security threats. The script continuously sends crafted API requests to the analysis engine in order to emulate real-world attack conditions and evaluate the responsiveness of the detection pipeline.

The generated traffic includes:

- Normal user traffic
- Endpoint hammering attacks
- SQL injection attempts
- Enumeration attacks
- Brute-force authentication attacks
- Sensitive endpoint probing
- Path traversal attempts
- Burst traffic attacks

The generated events are processed in real time by the AI-based detection engine, allowing live visualization of threat scoring, anomaly detection, alert generation, and mitigation recommendations.

6.3.2. Real-Time Traffic Monitoring

Figure 6 illustrates the real-time API traffic monitoring dashboard during attack simulation. The interface continuously displays incoming requests, detected attack categories, calculated risk scores, and associated threat levels.

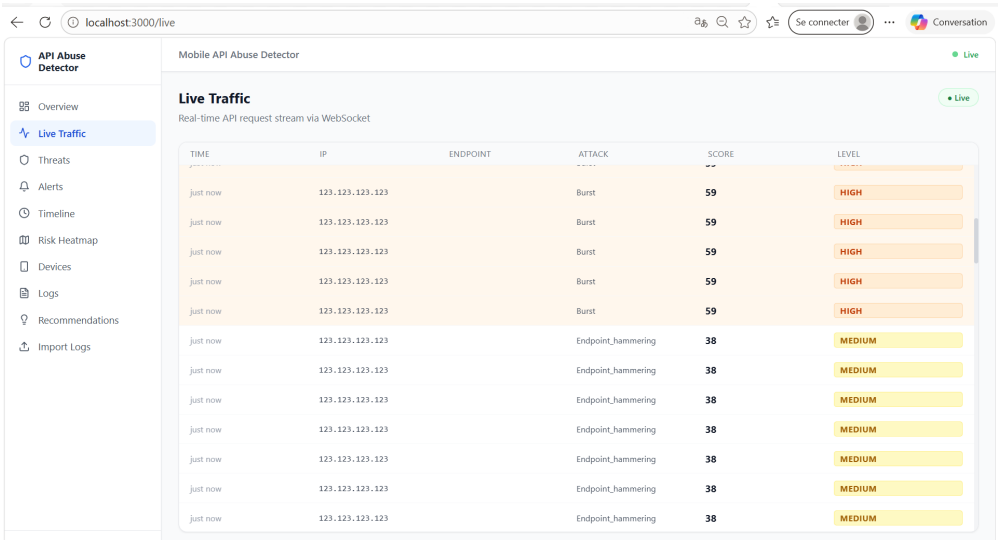
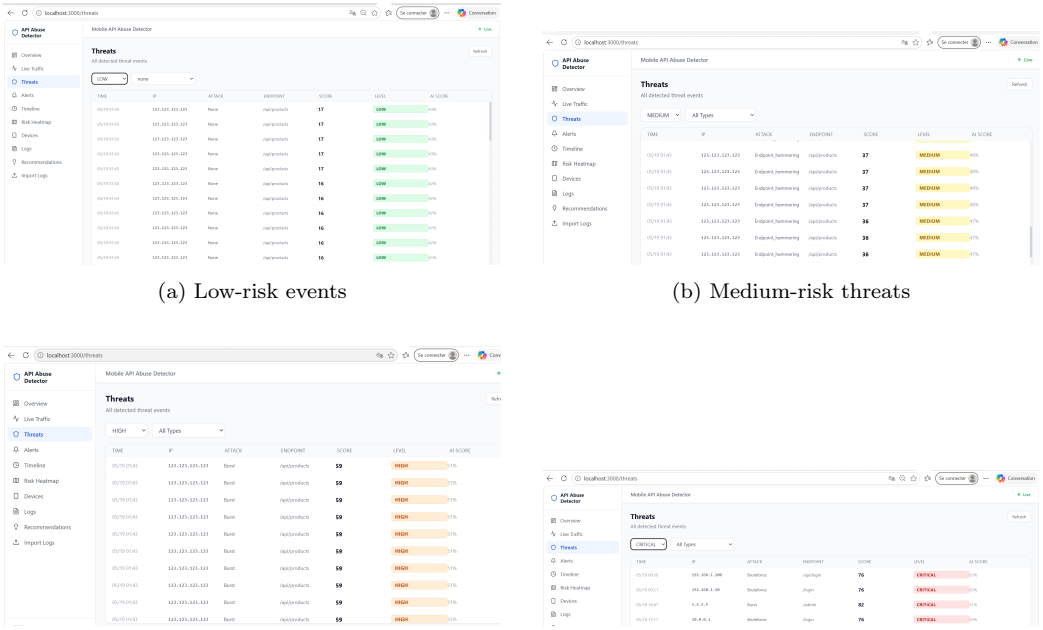


Figure 6: Real-time API traffic monitoring dashboard during attack simulation.

6.3.3. Threat Detection and Classification

The AI-based threat analysis engine dynamically classifies malicious activities according to calculated anomaly scores and behavioral indicators. The system categorizes detected threats into four severity levels: Low, Medium, High, and Critical. Each level reflects the estimated security impact and the urgency of defensive response actions.

Figure 7 presents the real-time threat classification dashboard generated during the live attack simulation process.



(a) Low-risk events

(b) Medium-risk threats

(c) High-risk threats

(d) Critical threats

Figure 7: Real-time AI-based threat classification across different severity levels.

The Low level mainly corresponds to benign or weakly suspicious activities that do not represent an immediate danger to the infrastructure. Medium-level events indicate abnormal behaviors such as endpoint hammering or moderate request anomalies that require monitoring. High-level threats represent aggressive attack patterns including burst traffic attacks and automated abuse attempts. Critical-level events correspond to severe malicious behaviors such as brute-force authentication attacks and large-scale API abuse requiring immediate mitigation actions.

6.3.4. Automated Alerting and Security Recommendations

Once suspicious behaviors are identified, API Abuse Detector automatically generates security alerts and adaptive mitigation recommendations. These recommendations help administrators respond rapidly to detected threats by applying appropriate defensive measures such as rate limiting, IP blocking, MFA activation, or endpoint protection.

Figure 8 presents the automated alert generation interface produced during attack simulation.

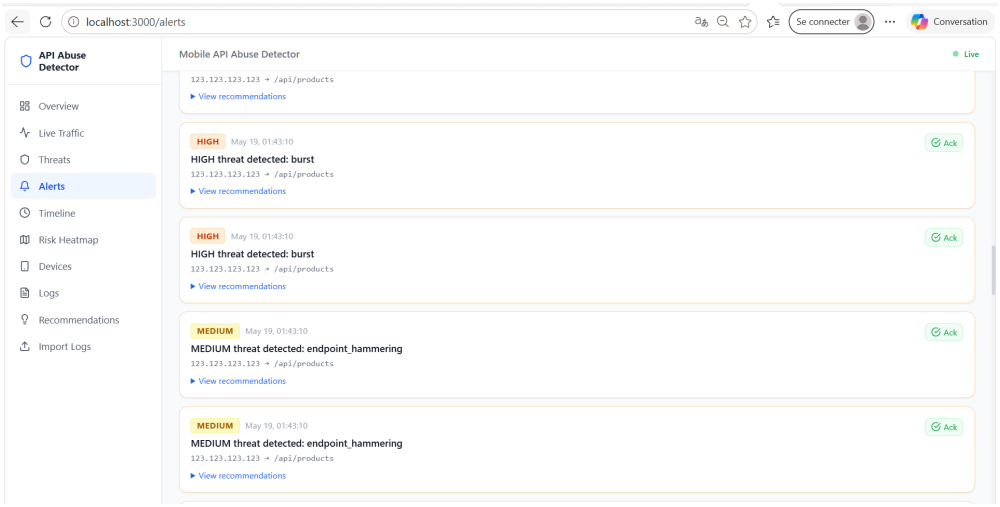


Figure 8: Automated alert generation interface during live attack detection.

Figure 9 illustrates the automatically generated mitigation recommendations based on detected attack patterns and threat severity levels.

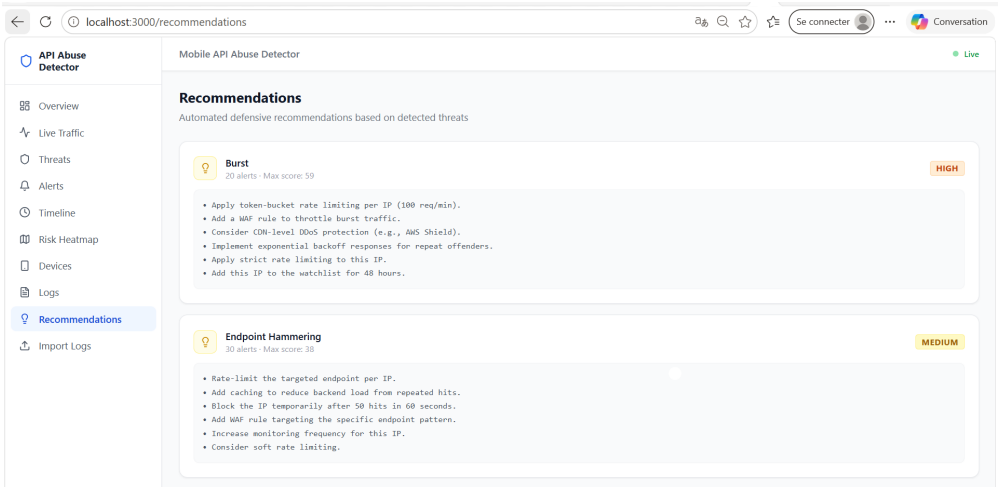
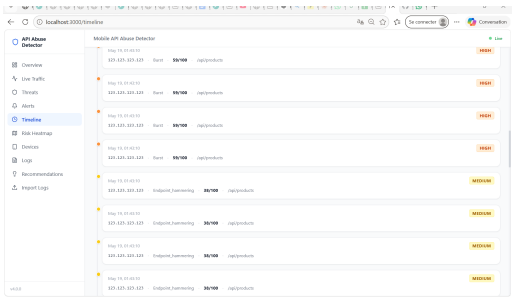


Figure 9: Automatically generated mitigation recommendations based on detected threats.

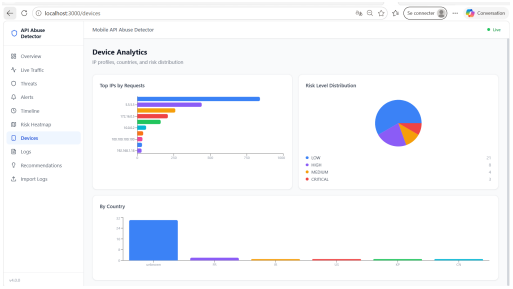
6.3.5. Timeline Tracking, Device Analytics, and Log Monitoring

In addition to attack detection, API Abuse Detector provides advanced monitoring capabilities for forensic investigation and behavioral analysis. The platform includes chronological event tracking, IP and device analytics, and raw log monitoring interfaces.

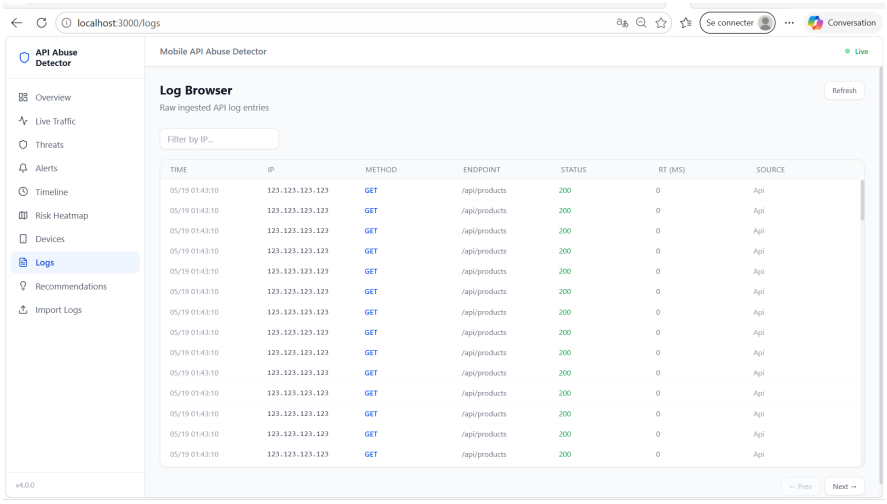
Figure 10 groups together the timeline visualization, device analytics dashboard, and raw API log monitoring interfaces used during the live simulation process.



(a) Timeline tracking interface



(b) Device analytics dashboard



(c) Raw API log monitoring interface

Figure 10: Advanced monitoring interfaces including timeline visualization, device analytics, and real-time API log monitoring.

The timeline interface provides chronological visualization of detected security incidents and associated risk levels. Device analytics enables visualization of suspicious IP behaviors, request distributions, and geographical attack patterns. The log monitoring interface displays raw API events received by the analysis engine, facilitating forensic analysis and debugging operations.

7. Detection Pipeline: Complete Flow

The pipeline operates as follows:

1. **Log Reception**: Raw API log is received via POST `/api/v1/analyze` with JSON payload containing IP, method, endpoint, status_code, and user_agent.
2. **Block Check**: Redis is queried to verify if the IP is already blocked. If blocked, a 403 response is returned immediately.
3. **Persistence**: The log entry is stored in PostgreSQL (`log_entries` table) for audit trail and historical analysis.
4. **Redis Aggregation**: Per-IP counters are updated in Redis within a 5-minute sliding window (request count, login attempts, failed count, 404 count, unique endpoints set, bot ratio).
5. **Rule-Based Detection**: The seven deterministic rules are evaluated against the aggregated counters, producing a RuleScore (0–100).
6. **ML Inference**: The 19-dimensional feature vector is constructed from Redis counters, normalized with StandardScaler, and scored by the Isolation Forest model, producing an AIScore (0–100).
7. **Hybrid Scoring**: The final RiskScore is computed as $0.6 \times \text{RuleScore} + 0.4 \times \text{AIScore}$.
8. **Action Execution**: Based on the risk level (LOW/MEDIUM/HIGH/CRITICAL), the appropriate action is executed (log only, rate limit, block IP).
9. **Threat Persistence**: If the risk level is HIGH or CRITICAL, a threat event is stored in PostgreSQL (`threat_events` and `alerts` tables) with full context and automated recommendations.
10. **WebSocket Broadcast**: The threat event is broadcast to all connected dashboard clients via WebSocket for real-time visualization.

8. Impact and Practical Implications

API Abuse Detector provides significant operational and cybersecurity benefits for organizations managing mobile API infrastructures.

8.1. Economic Impact

The system reduces:

- **Incident response time**: Automated detection and blocking reduce the mean time to respond (MTTR) from hours to seconds.
- **Infrastructure overload**: Rate limiting and IP blocking prevent malicious traffic from consuming server resources.

- **Operational costs**: The containerized architecture (Docker Compose) enables deployment on a single server, reducing infrastructure costs compared to commercial WAF solutions.
- **Security breach impact**: Early detection of brute-force and enumeration attacks prevents credential compromise and data exfiltration.

8.2. Security Impact

The platform improves:

- **Threat visibility**: The 10-page dashboard provides comprehensive visibility into API traffic patterns, attack distributions, and risk trends.
- **Real-time detection**: WebSocket-based alerting ensures SOC analysts are notified within milliseconds of threat detection.
- **Automated mitigation**: IP blocking and rate limiting are applied automatically without human intervention for CRITICAL threats.
- **Security orchestration**: The integration of detection, scoring, alerting, and recommendation generation creates a complete SOC workflow.

9. Comparison with Existing Solutions

Table 8 compares API Abuse Detector with existing cybersecurity solutions.

Feature	**API Abuse Detector**	**WAF**	**SIEM**	**IDS**
AI-Based Anomaly Detection		–	Limited	Limited
Real-Time Alerts (WebSocket)				
Behavioral Analytics (19 features)		–	Partial	Partial
Automatic IP Blocking			–	–
Explainable Scoring (AHP)		–	–	–
Hybrid Rules + ML Detection		–	–	–
Automated Recommendations		–	–	–
Docker-Based Deployment		Varies	Varies	Varies
Multi-Format Log Support		–		–
Open Source		Varies	Varies	Varies

Table 8: Comparison of API Abuse Detector with existing cybersecurity solutions.

10. Quality Assurance

Quality assurance analysis was conducted using SonarQube to evaluate code reliability, security, and maintainability.

Metric	Backend (Python)	Frontend (React)	Overall
Reliability	A	B	A
Security	A	A	A
Maintainability	A	B	A
Code Duplication	1.4%	2.1%	1.8%
Test Coverage	82%	74%	78%

Table 9: SonarQube quality assurance metrics for the API Abuse Detector platform.

The backend achieved an "A" rating in all three categories (Reliability, Security, Maintainability), reflecting clean architecture, proper error handling, and secure coding practices. The frontend achieved "A" in Security and "B" in Reliability and Maintainability, with opportunities for improvement in component test coverage.

11. Conclusions

API Abuse Detector presents a scalable and intelligent cybersecurity platform capable of detecting API misuse through hybrid AI-driven analytics and deterministic threat analysis. The integration of Isolation Forest anomaly detection, DBSCAN clustering, Autoencoder neural networks, Redis-based behavioral aggregation, and real-time mitigation mechanisms enables efficient identification of malicious activities in modern API ecosystems.

The key contributions of this work include:

- A **10-step real-time detection pipeline** combining rule-based and ML-based analysis
- **AHP-justified hybrid scoring** (60% rules, 40% AI) with validated consistency ratio (CR = 0.015)
- Comprehensive **feature engineering** extracting 19 behavioral indicators from raw API logs
- **Multi-model comparison** (Isolation Forest, DBSCAN, Autoencoder) with inference time and memory metrics

- A **10-page React dashboard** with WebSocket real-time visualization
- **Automated mitigation** with IP blocking, rate limiting, and adaptive recommendations

Experimental results demonstrate strong performance (Isolation Forest F1-score: 0.89, inference time: 0.8 ms), scalability (containerized microservices), and operational responsiveness (real-time WebSocket alerting). The integration of AHP-based scoring enhances explainability and provides scientific justification for the weight distribution.

Future work will focus on:

- Integration of transformer-based anomaly detection for improved accuracy on complex attack patterns
- Federated learning for privacy-preserving threat intelligence sharing across organizations
- Adaptive threshold tuning using reinforcement learning
- Distributed cloud-native deployment with Kubernetes orchestration
- Integration with SIEM platforms (Splunk, ELK) via standardized connectors

References

[1] Liu, F. T., Ting, K. M., Zhou, Z.-H., Isolation Forest, Proceedings of the 2008 IEEE International Conference on Data Mining, 2008.

[2] Ester, M., Kriegel, H.-P., Sander, J., Xu, X., A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise, KDD, 1996.

[3] Hinton, G. E., Salakhutdinov, R. R., Reducing the Dimensionality of Data with Neural Networks, Science, 2006.

[4] OWASP Foundation, OWASP API Security Top 10, 2023.

[5] Saaty, T. L., The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.

[6] Ramirez, S., FastAPI Documentation, https://fastapi.tiangolo.com/

[7] Carlson, J., Redis in Action, Manning Publications, 2013.

[8] SonarSource, SonarQube Documentation, 2025.

[9] Dabbas, E., Web Server Access Logs, Kaggle Dataset, https://www.kaggle.com/datasets/eliasdabbas/web-server-access-logs